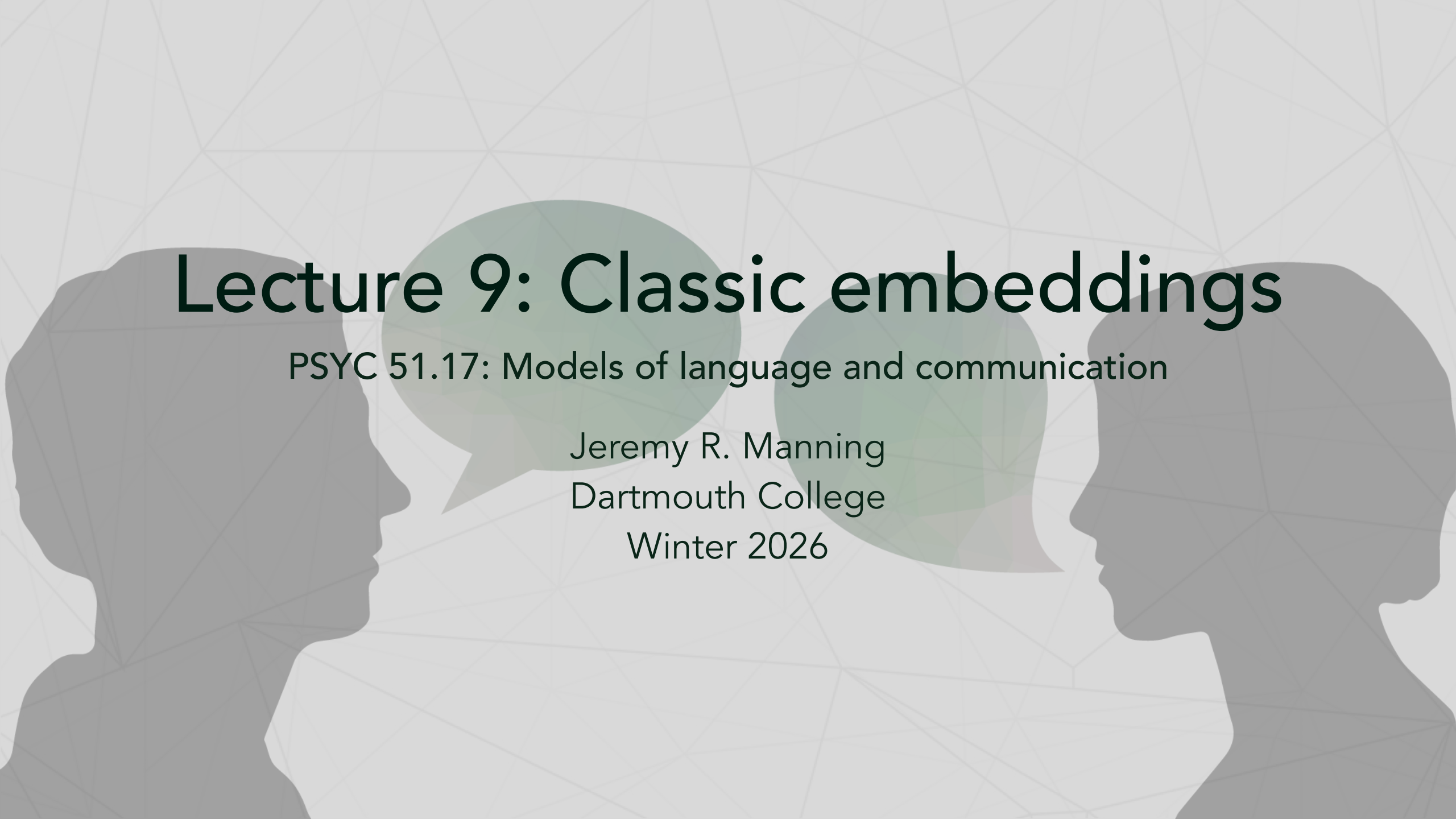




Lecture 9: Classic embeddings

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will

1. Understand the distributional hypothesis and why it matters
2. Implement Latent Semantic Analysis (LSA) with SVD
3. Apply Latent Dirichlet Allocation (LDA) for topic modeling
4. Compare classic embedding methods and their trade-offs
5. Visualize and interpret semantic relationships

Central question

"You shall know a word by the company it keeps" — J.R. Firth (1957)

The fundamental question

How do we represent the meaning of words computationally?

Traditional Approach:

- Dictionaries
- Taxonomies (WordNet)
- Manual feature engineering

Labor-intensive, hard to scale

Distributional Approach:

- Learn from data
- Context-based
- Scalable, unsupervised

Let the data tell us what words mean!

The distributional hypothesis

Key Idea: Words that appear in similar contexts tend to have similar meanings.

Example

- "The **cat** sat on the mat"
- "The **dog** sat on the mat"
- "The **kitten** sat on the mat"

→ *cat, dog, kitten* are semantically related

Implication

We can learn word meaning from co-occurrence patterns alone!

From words to vectors

Goal: Represent each word as a point in high-dimensional space

Word-context co-occurrence

Word	with "the"	with "sat"	with "ran"
cat	2	1	0
dog	2	0	1
mat	1	1	0

Problem: Real vocabularies have 10,000+ words!

→ We need **dimensionality reduction**

Latent Semantic Analysis (LSA)

The OG of semantic embeddings (1990)

Algorithm:

1. Build term-document matrix X
2. Apply TF-IDF weighting (recall from last week!)
3. Perform SVD: $X = U\Sigma V^T$
4. Keep top k dimensions
5. Use U_k as word embeddings

SVD Decomposition:

- | | |
|---|-------------------------------------|
| 1 | $X = U \times \Sigma \times V^T$ |
| 2 | |
| 3 | U : word-topic associations |
| 4 | Σ : topic strengths |
| 5 | V^T : topic-document associations |

Key innovation

Discovers latent topics, solves synonymy problem, captures semantic similarity

LSA in Python

```
1 from sklearn.decomposition import TruncatedSVD
2 from sklearn.feature_extraction.text import TfidfVectorizer
3
4 documents = [
5     "The cat sat on the mat",
6     "The dog ran in the park",
7     "Machine learning is amazing",
8 ]
9
10 # Build TF-IDF matrix (recall from week 2!)
11 tfidf = TfidfVectorizer(max_features=5000, stop_words='english')
12 tfidf_matrix = tfidf.fit_transform(documents)
13
14 # Apply LSA - reduce to 100 dimensions
15 lsa = TruncatedSVD(n_components=100, random_state=42)
16 doc_embeddings = lsa.fit_transform(tfidf_matrix)
17
18 print(f"Original: {tfidf_matrix.shape}") # (3, vocab_size) - sparse
```

continued...

LSA in Python

```
19 | print(f"After LSA: {doc_embeddings.shape}") # (3, 100) - dense!
```

...continued

Finding similar words with LSA

```
1 from sklearn.metrics.pairwise import cosine_similarity
2
3 # Get word embeddings
4 vocab = tfidf.get_feature_names_out()
5 word_embeddings = lsa.components_.T
6
7 def find_similar(word, top_n=5):
8     idx = list(vocab).index(word)
9     word_vec = word_embeddings[idx].reshape(1, -1)
10    sims = cosine_similarity(word_vec, word_embeddings)[0]
11    top_indices = sims.argsort()[-top_n:-1][::-1]
12    return [(vocab[i], f"{sims[i]:.3f}") for i in top_indices]
```

Key insight

LSA finds semantically related words even without explicit labels!

LSA limitations

Issues:

- SVD is expensive for large corpora
- Hard to update with new documents
- Linear relationships only
- No polysemy handling

Example: Polysemy

Word: "bank"

1. Financial institution
2. River bank
3. Banking (airplane)

LSA gives ONE vector for all meanings!

Latent Dirichlet Allocation (LDA)

A probabilistic approach to topic modeling

Key assumptions

1. Each document is a mixture of topics
2. Each topic is a distribution over words
3. The mixture proportions vary per document

Advantages over LSA:

- Probabilistic interpretation
- Non-negative weights
- More interpretable topics

Example: Doc about "tech pets":

- 60% Technology topic
- 30% Animals topic
- 10% Other

LDA in Python

```
1 from sklearn.decomposition import LatentDirichletAllocation
2 from sklearn.feature_extraction.text import CountVectorizer
3
4 # Create bag-of-words (not TF-IDF for LDA!)
5 vectorizer = CountVectorizer(max_features=1000, stop_words='english')
6 doc_term_matrix = vectorizer.fit_transform(documents)
7 vocab = vectorizer.get_feature_names_out()
8
9 # Fit LDA
10 lda = LatentDirichletAllocation(n_components=5, random_state=42)
11 doc_topics = lda.fit_transform(doc_term_matrix)
12
13 # Print top words per topic
14 for idx, topic in enumerate(lda.components_):
15     top_words = [vocab[i] for i in topic.argsort()[-7:]]
16     print(f"Topic {idx}: {' '.join(top_words)}")
```

LDA topic examples

Example Topics from News Corpus:

Topic 1: Politics (20%)

government, election, president, vote, policy, senate

Topic 2: Sports (15%)

game, team, player, score, win, season, coach

Topic 3: Technology (25%)

software, computer, data, algorithm, system, AI

Document representation: [0.20, 0.15, 0.25, 0.40] ← topic mixture

LSA vs. LDA comparison

Aspect	LSA	LDA
Approach	Matrix factorization	Probabilistic
Interpretability	Medium	High
Speed	Fast	Medium
Topic coherence	Medium	High
Best for	Search, retrieval	Topic discovery

Visualizing embeddings with UMAP

Project high-dimensional vectors to 2D:

```
1 import umap
2
3 # Reduce from 100D to 2D
4 reducer = umap.UMAP(n_neighbors=15, min_dist=0.1, metric='cosine')
5 embeddings_2d = reducer.fit_transform(word_embeddings)
6
7 # Plot - similar words cluster together!
8 plt.scatter(embeddings_2d[:, 0], embeddings_2d[:, 1])
9 for i, word in enumerate(words_to_plot):
10     plt.annotate(word, (embeddings_2d[i, 0], embeddings_2d[i, 1]))
```

What you should see

Sports words cluster together, tech words cluster together, etc. — even though we never told the model about categories!

Discussion: Does this capture "meaning"?

Arguments For:

- Captures semantic similarity
- Works well in practice
- Aligns with linguistic theory
- Scalable to large corpora

Arguments Against:

- Text-only (no grounding)
- Missing embodied experience
- No common sense reasoning
- Reflects biases in data

The Symbol Grounding Problem

"How can meaning be intrinsic to the system, rather than parasitic on the meanings in our heads?" — Harnad (1990)

Summary

What we learned:

1. **Distributional Hypothesis:** Words in similar contexts have similar meanings
2. **LSA:** SVD on term-document matrix → dense embeddings
3. **LDA:** Probabilistic topic modeling → interpretable topics
4. **Visualization:** UMAP reveals semantic structure

Limitation

These are **static** embeddings — one vector per word, no context!

Tomorrow: Neural word embeddings (Word2Vec, GloVe, FastText)

Key references

Classic Papers:

- Deerwester et al. (1990). "Indexing by Latent Semantic Analysis"
- Blei et al. (2003). "Latent Dirichlet Allocation"

Tools:

- Scikit-learn: `TruncatedSVD` , `LatentDirichletAllocation`
- Gensim: `LsiModel` , `LdaModel`

Try it out!

Check out the [Topic Modeling demo](#)

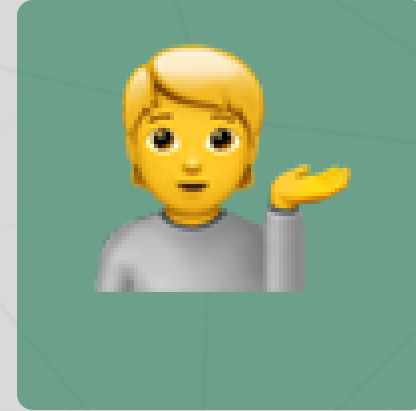
Questions?



Email me



Join our Discord



Come to office hours

See you tomorrow!

Friday we dive into neural word embeddings!